Trade Intelligence Copilot

A Deep Dive: agentic retrieval over trade data,
with an evaluation layer that is the point

Explainer for Salman Adnan's portfolio

Read this first: what this document assumes

Nothing. This document assumes you have never built a retrieval system, never written a line of SQL, and have never heard the phrase "agentic retrieval". Every technical term is defined in a **Jargon** box the first time it appears. If you already know the vocabulary, skip the boxes and read the diagrams.

Contents

1 What this project is, in one paragraph **3**

2 The ideas, from first principles **3**
2.1 Large language models, and their central weakness . . . 3
2.2 Retrieval-augmented generation (RAG) . . . 3
2.3 Agentic retrieval: the step that makes it "agentic" . . . 4
2.4 Text-to-SQL . . . 4
2.5 Embeddings, vectors and cosine similarity . . . 5
2.6 Chunks and citation . . . 5
2.7 An evaluation layer, and why it is the whole point . . . 5

3 Map of the repository **6**

4 End-to-end architecture **8**

5 The data layer **8**
5.1 Where the trade data comes from . . . 8
5.2 The schema . . . 9
5.3 The scope facts: the quiet foundation of honest refusal . . . 9
5.4 The regulations corpus . . . 10
5.5 Ingestion: chunking and embedding . . . 10

6 Embeddings and the vector store **11**
6.1 The retrieval path . . . 11
6.2 The store itself . . . 11
6.3 Why embeddings go through raw REST . . . 12

7 The planner: the agentic step **12**

8 The three tools **13**
8.1 Tool 1: text-to-SQL . . . 13
8.2 Tool 2 and 3, briefly . . . 14
8.3 The SQL guard: the security boundary . . . 14

9 Refusal, synthesis and citation **16**
9.1 The deterministic refusal . . . 16
9.2 How refusal actually happened in the real run . . . 16
9.3 Synthesis . . . 17
9.4 Citations, and one latent problem . . . 17

10 Cost accounting, the server and the UI **18**

11 The evaluation layer **19**
11.1 The test set . . . 19
11.2 The harness . . . 19
11.3 How each metric is computed . . . 20
11.4 The published results . . . 20

12 What I verified by actually running it **20**
12.1 The gold answers are genuinely ground truth . . . 21
12.2 The metrics reproduce exactly . . . 21
12.3 The test suite . . . 21
12.4 So is "15/15" verified? . . . 21
12.5 What the two retrieval misses actually were . . . 21
12.6 The one unfaithful verdict, read in full . . . 22

13 Consolidated honest findings **22**
13.1 Findings the project already owns, and gets right . . . 22
13.2 Findings from this review . . . 23
13.3 Limitations the project states, and should keep stating . . . 23

14 Defending this project in an interview **24**
14.1 Questions you should expect . . . 24

15 Glossary **24**

1 What this project is, in one paragraph

Trade Intelligence Copilot answers plain-English questions about Pakistani sugar-sector import data and about trade-regulation concepts. You type "Which country was the largest source of imports by value?" and it replies with a number taken from a real database, showing you the exact query it ran and the rows it got back. You type "What is a phytosanitary certificate?" and it replies with a definition drawn from a specific document, and tells you which document. You type "What were Pakistan's wheat exports to India in 2019?" and it says: *this question is not supported by the available data*, because the dataset holds no exports, no wheat, and no 2019.

That last behaviour is the hard one, and it is why the project exists.

The thesis of the project

Most demonstrations of this technology answer confidently and are never measured. This one ships a 28-item test set with ground-truth answers computed directly from the database, runs the whole system against a live model API, publishes the real numbers, and is built to check its own automatic grader against hand labels (redone per run; pending for the current one). The headline is not the copilot. The headline is the **evaluation layer**.

2 The ideas, from first principles

Before any code, here is the vocabulary. Each idea is small; the system is just these ideas wired together.

2.1 Large language models, and their central weakness

Jargon: Large language model (LLM)

A program trained on an enormous amount of text to predict what text should come next. Given "The capital of France is", it answers "Paris". Modern ones follow instructions, write code, and produce structured output. This project uses Google's Gemini models through Vertex AI, Google's cloud platform for machine-learning services.

An LLM knows only what it absorbed during training. It does not know what is in *your* database. Worse, when it does not know, it often invents an answer that reads perfectly. This is called **hallucination**, and for a system that reports trade values in dollars it is fatal: a confidently invented $175 million is worse than no answer at all.

Jargon: Hallucination

When a language model states something fluent, plausible and false. The model is not lying; it has no concept of truth. It is producing text that pattern-matches what a correct answer looks like.

2.2 Retrieval-augmented generation (RAG)

The standard fix is to stop asking the model to *remember* and start forcing it to *read*.

Jargon: Retrieval-augmented generation (RAG)

A pattern in two beats. First **retrieve**: go and fetch the specific facts or documents relevant to the question from a trusted source. Then **generate**: hand the model *only* those facts and instruct it to answer using nothing else. The model contributes fluency and reasoning; the

trusted source contributes the facts.

This is the difference between a closed-book exam and an open-book exam where you are only allowed to cite the page in front of you.

2.3 Agentic retrieval: the step that makes it "agentic"

Plain RAG always does the same thing: search a pile of documents, paste the results in, answer. But the questions here are not all the same shape. "What was the total USD value of all imports?" is not a question about documents at all; it is arithmetic over a database. "What is a certificate of origin?" is purely a document question. "How much sugar was imported, and why is sugar policy-sensitive?" is both at once.

Jargon: Agentic retrieval

Retrieval where the system first **decides for itself** which sources to consult, and how many, rather than always running one fixed search. A model reads the question and picks the tools. That decision step is what earns the word "agentic". The system is choosing a strategy, not executing a fixed recipe.

Jargon: Tool

A concrete capability the system can invoke: run a database query, search documents, look up a code. The model does not perform the tool's work; it *names* the tool it wants and ordinary code performs the work. This distinction matters for safety: the model never touches the database directly.

In this project the planner is an LLM whose entire job is to answer: does this question need SQL, document retrieval, an HS-code lookup, or some combination? It is explicitly told *not* to answer the question itself.

2.4 Text-to-SQL

Jargon: SQL

Structured Query Language, the standard language for asking questions of a relational database (data in tables of rows and columns). `SELECT COUNT(*) FROM trade_transactions` means "count every row in the trade_transactions table". `SELECT` reads data; other commands such as `INSERT`, `UPDATE`, `DELETE` and `DROP` change or destroy it.

Jargon: Text-to-SQL

Having a language model translate a plain-English question into a SQL query, then running that query on the real database and using the returned rows as the answer. The model supplies the translation; the database supplies the number.

This is the single most important idea for trustworthiness here. When you ask for a total, the model is never asked to recall or estimate the total. It is asked to write a query. The database computes the number. If the query is right, the number is right by construction.

The obvious danger: the model is writing executable code against your database, and a model can be talked into writing `DROP TABLE`. Section 8.3 is entirely about containing that.

2.5 Embeddings, vectors and cosine similarity

For the document half of the system, we need to find text that is *about* the same thing as the question, even when it shares no words with it. Searching for the literal word "phytosanitary" is brittle; the user might ask about "plant health paperwork".

Jargon: Embedding

A list of numbers (here, 768 of them) that represents the *meaning* of a piece of text, produced by a model trained for the purpose. The useful property: texts with similar meanings get similar lists, even with no words in common.

Jargon: Vector

A list of numbers treated as a point, or an arrow from the origin, in space. 768 numbers means a point in 768-dimensional space. You cannot picture it; the arithmetic works exactly as it does in the two dimensions you can picture.

Jargon: Cosine similarity

A measure of whether two vectors point the same way, ignoring their length. It runs from 1.0 (identical direction, so: same meaning) through 0 (unrelated) to −1.0 (opposite). "Similar meaning" becomes "small angle between the arrows", and that is a number you can sort by.

The trick that makes this cheap: if every vector is first scaled to length exactly 1 ("normalized"), then cosine similarity is just the dot product, which is one multiply-and-add per dimension. The project exploits this directly, and Section 6 shows the four lines of code where it happens.

2.6 Chunks and citation

Jargon: Chunk

A document cut into a retrievable piece. You do not embed a whole document, because then a hit tells the reader "the answer is somewhere in these five pages". This project splits each document at its `##` section headings, so one chunk is one whole section, which is also the natural thing to cite.

Each chunk carries an id like `phytosanitary-sps-basics#s2`, meaning section 2 of that document. When the copilot states a rule, it prints that id. The claim is auditable: you can go read the exact section it came from.

2.7 An evaluation layer, and why it is the whole point

Jargon: Evaluation layer ("eval")

Machinery that measures whether the system is actually right, on a fixed set of questions with known correct answers, producing numbers you can publish and argue with. Not a demo, not a vibe check: a score.

Jargon: Ground truth

The known-correct answer, established independently of the system being tested. Here, the gold answer to "how many shipments came from China?" is not a human guess: it is the result of running the counting query against the database directly. It cannot be wrong, so

"correct" stops being a matter of opinion.

Why an eval changes the argument

Without an eval you can say "it seems to work". An interviewer cannot check that, and neither can you. With an eval you can say "on 15 questions whose answers I computed from the database, it got 15 right, and here is the harness". It also catches bugs no amount of looking at pretty answers ever would: this project's text-to-SQL step once produced a query that returned zero rows, and the answer that reported "0 shipments" looked completely reasonable. Only the gold number, which was dozens, exposed it.

Jargon: LLM-as-judge

Using a second, usually stronger, language model to grade the first one's output. Here `gemini-2.5-pro` reads the answer plus the evidence and rules on whether every claim is supported. Necessary because "did this paragraph invent anything?" cannot be checked with string matching.

Jargon: Faithfulness

Whether an answer is fully supported by the evidence it was given. Note carefully that faithfulness is **not** correctness. An answer that faithfully reports the output of a wrong query is faithful and wrong. This project keeps the two measurements separate on purpose, and Section 13 explains why that separation was forced on it by a real bug.

A judge you have not validated is not evidence

If you grade a model with a model, who grades the judge? This project answers that: the author hand-labels a 12-item subset, and the harness reports how often the judge agreed with the author (11 of 12 on the first version; pending for the current run). That agreement number is what makes the faithfulness number mean anything at all.

3 Map of the repository

Every tracked file, and what it is for.

```
trade-intelligence-copilot
├─ app/  the copilot itself
│   ├─ config.py  all settings + secrets read from environment
│   ├─ llm.py  Gemini wrapper + token-cost meter
│   ├─ embeddings.py  Vertex embedding REST calls
│   ├─ vectorstore.py  numpy cosine-similarity search
│   ├─ schema_prompt.py  DB schema + scope facts + SQL rules
│   ├─ planner.py  THE AGENT: picks the tools
│   ├─ tools.py  text-to-SQL, retrieval, hs_ lookup
│   ├─ db.py  SQLite access + the SQL safety guard
│   ├─ answer.py  grounded synthesis + refusal
│   ├─ agent.py  orchestration, trace, cost
│   ├─ server.py  FastAPI: /ask, /health, /
│   └─ templates/index.html  single-page UI
├─ data/
│   ├─ build_db.py  seeded generator -> trade.sqlite
│   ├─ trade.sqlite  2,000 synthetic shipments (prebuilt, shipped)
│   ├─ regulations/  16 authored explainer documents
│   ├─ chunks.json  48 chunks + metadata (prebuilt)
│   └─ vectors.npz  48x768 normalized matrix (prebuilt)
├─ ingest/ingest.py  chunk + embed the corpus
├─ eval/  THE HEADLINE
│   ├─ testset.jsonl  28 questions + gold answers
│   ├─ run_eval.py  harness: run, judge, score
│   ├─ transcripts.json  every answer from the real run
│   ├─ results.json  the published metrics
│   └─ human_labels.json  hand labels for the judge check (pending)
├─ tests/  33 tests: 29 offline, 4 live
├─ docs/  ARCHITECTURE.md, EVALUATION.md
├─ requirements.txt
└─ .env.example  config template, no real values
```

Figure 1: The repository. Roughly 1,000 lines of Python, deliberately small: the substance is in `eval/`, not in framework scaffolding.

No framework

There is no LangChain, no LlamaIndex, no agent framework. The planner, the tool dispatch, the vector store and the eval harness are all hand-written and total about a thousand lines. This is a defensible choice for a project whose purpose is to demonstrate that the author understands the mechanism: a framework would hide exactly the parts worth showing.

4 End-to-end architecture

Here is the entire system. One question enters at the left; an answer with citations, a trace, a cost and a latency leaves at the right.

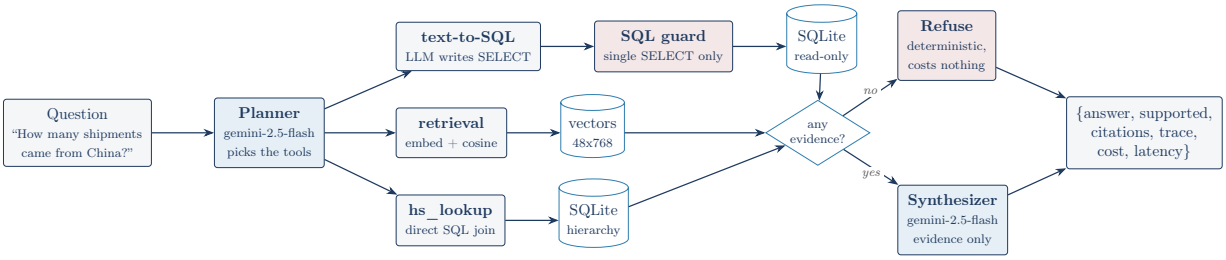


Figure 2: The agent loop. Three model calls on a typical answered question (plan, text-to-SQL, synthesize); two on a refusal, because the deterministic check fires before the expensive synthesis call.

Read that diagram as a sentence: *an LLM decides which tools to run; ordinary code runs them; a cheap check refuses when they returned nothing; and a second LLM writes prose using only what they returned.*

Where the trust lives

Notice what the models are and are not allowed to do. The planner chooses but does not answer. The text-to-SQL model writes a query but never executes it: the guard and the read-only connection do that. The synthesizer writes prose but is handed a fixed block of evidence and told to use nothing else. No model is ever trusted with both the facts and the final word.

5 The data layer

5.1 Where the trade data comes from

`data/build_db.py` *generates* the trade data: a compact SQLite database of synthetic Pakistani sugar-sector imports. Every importer, supplier, shipping agent, price and quantity is made up. Only the public scaffolding is real: the HS chapter 17 headings and the country names. The first version of the project ran on a dataset that cannot be published, and this generator replaced it.

Jargon: SQLite

A complete relational database that is just a single file on disk, with no server to install or run. Ideal here: the demo database ships inside the repo and the project runs with nothing to set up.

Jargon: Seeded generator

A program that draws "random" values from a pseudo-random number generator started at a fixed number, the seed. Same seed, same sequence, so the output is identical on every run. Here `SEED = 1717`, and two builds of `trade.sqlite` are byte-for-byte the same file.

The script is deliberately plain: drop the file if it exists, create the schema, write the HS hierarchy, then draw 2,000 shipments. Each shipment picks a subcategory by weight, an origin country (bulk sugar and molasses only from Brazil, Thailand or India, weighted so Brazil clearly leads by value), a price from that subcategory's band, and a log-uniform quantity, so there are many small lots and a few large ones. Determinism is what makes the evaluation honest after the switch: `eval/build_testset.py` recomputes every gold answer from the generated file with a query, so nobody types a gold number by hand.

5.2 The schema

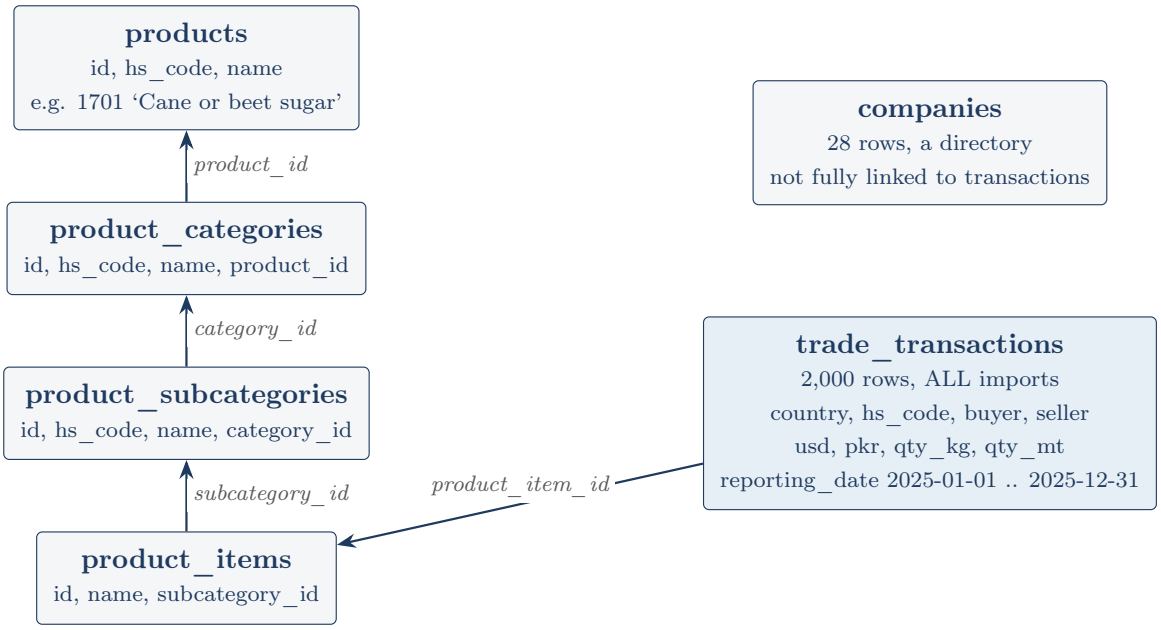


Figure 3: The HS-code hierarchy on the left, the fact table on the right. Four indexes exist on `trade_transactions`: country, hs_code, trade_type and reporting_date, which are exactly the columns the questions filter on.

Jargon: HS code

The Harmonized System, an international product-classification standard. The first two digits are the chapter, the first four the heading. This dataset is entirely chapter 17: 1701 cane and beet sugar, 1702 other sugars and syrups, 1703 molasses, 1704 sugar confectionery. Codes are stored as *text*, not numbers, so that `1702.1100` keeps its shape and prefix matching works.

5.3 The scope facts: the quiet foundation of honest refusal

`app/schema_prompt.py` carries three blocks of text that are injected into prompts. The second, `DATASET_SCOPE`, is the most interesting thing in the file, and it is easy to overlook:

```
DATASET_SCOPE = """
DATASET SCOPE (important for deciding if a question is answerable):
- trade_transactions contains 2000 rows. Every row is an IMPORT.
  There are NO export rows in this dataset.
```

```
- All shipments are in HS chapter 17 (sugar and sugar products).
  There is no data for other commodities (no rice, wheat, cotton...).
- reporting_date ranges from 2025-01-01 to 2025-12-31 only.
...
If a question asks about data outside this scope (exports, non-sugar
commodities, other years...), it CANNOT be answered from the data.
"""
```

Listing 1: The scope facts, given to both the planner and the text-to-SQL step (abridged).

Refusal starts here, not in the refusal code

A model cannot decline a question about exports unless it knows the dataset has no exports. These sentences are what let the planner recognise "Pakistan's wheat exports in 2019" as three separate impossibilities at once. Every one of these facts is checkable against the database, and I checked all of them (Section 12): 2,000 rows, `trade_type` is `'import'` for every row, dates run exactly 2025-01-01 to 2025-12-31, 28 companies. The docstring calls them "computed directly from the database", and that is true.

The third block, `TEXT_TO_SQL_RULES`, is where a real bug was fixed. It now insists on wildcard matching for any name or free-text column, and forbids matching a name with `=`:

```
- Match names or free text with LIKE wrapped in % on both sides, for example
  buyer LIKE '%Tamrin%' or country LIKE '%South Africa%'. Never match a name with =.
```

Listing 2: One rule from TEXT_TO_SQL_RULES, added after the bug in Section 13. The example name is deliberately not one the evaluation asks about.

5.4 The regulations corpus

Sixteen short Markdown documents in `data/regulations/`, covering HS-code basics, phytosanitary and SPS rules, customs-duty concepts, import documentation, anti-dumping and safeguards, letters of credit, the Pakistan Single Window, and one document per HS heading 1701 to 1704.

These are authored summaries, not law, and the project says so

The corpus is plain-language explainers written for this demo. They reproduce no statute, no SRO number and no actual duty rate, and `_README.md` in that folder states this plainly, as does `docs/EVALUATION.md`. This is an honest and necessary disclosure: the faithfulness metric measures grounding in *this corpus*, not legal accuracy. A retrieval system that is 100% faithful to a made-up document is 100% useless for actual customs compliance. The project does not overclaim here, and it should not start.

5.5 Ingestion: chunking and embedding

`ingest/ingest.py` turns those documents into searchable vectors. Its cleverest function, `split_sections`, walks a document line by line. It treats every second-level Markdown heading (a line starting with two hash characters) as a chunk boundary, and captures the top-level heading as the document title.

ingest.run(), the whole pipeline

```
1.  refuse to start unless credentials are present
2.  for each *.md in regulations/, skipping files starting with "_":
    read it; find the "# " document title
    split on "## " headings into (section_title, section_text)
    skip the front-matter block (section title == doc title)
```

```
    chunk id = "<filename stem>#s<index>"
    embed_text = "<doc title> ::  <section title>\n<section text>"
3.  embed every embed_text in batches of 5 (RETRIEVAL_DOCUMENT)
4.  normalize all vectors to unit length
5.  write vectors.npz (matrix) + chunks.json (metadata, embed_text dropped)
```

Two details worth pausing on, because both are real craft:

- **The embedded text is not the stored text.** What gets embedded is `"<doc title> :: <section title>\n<section text>"`, so the document and section headings contribute to the meaning of the vector. A section called "2. Reading the digits" is nearly meaningless on its own; as "HS codes: structure and how classification works :: 2. Reading the digits" it is findable. What is *stored* and later shown to the reader is the plain section text.
- **The task type is set.** Vertex embedding models accept a `task_type`. Ingestion passes `RETRIEVAL_DOCUMENT`; a live query passes `RETRIEVAL_QUERY` (`app/tools.py`, line 51). The model embeds a question and a passage into the same space but with the asymmetry between them accounted for. Getting this backwards silently degrades every search, and it is a common mistake.

The result, verified: 48 chunks from 16 documents, each a 768-dimensional vector, cached on disk so that neither the API nor the eval ever re-embeds the corpus.

6 Embeddings and the vector store

6.1 The retrieval path

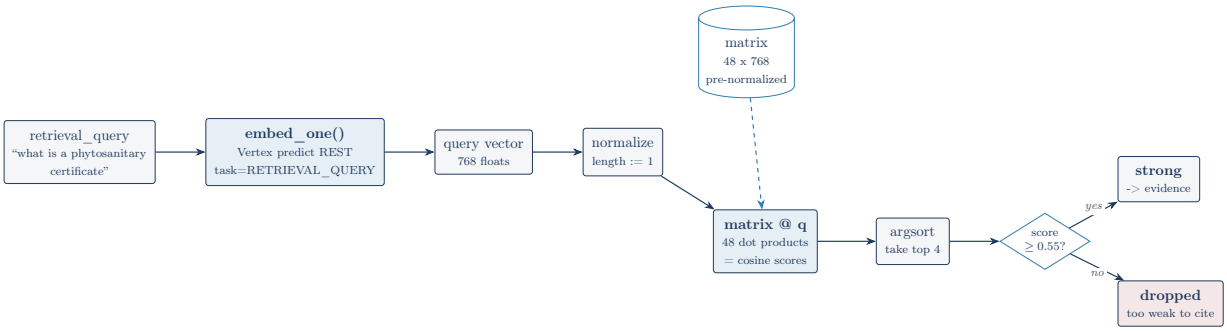


Figure 4: The retrieval path. The threshold at the end is what makes refusal possible: retrieval always returns its top 4, so "nothing relevant" has to be *defined*, and it is defined as "nothing scored 0.55 or better".

Why the threshold is load-bearing

A nearest-neighbour search never fails. Ask it about wheat exports and it will cheerfully hand back the four *least irrelevant* sugar documents, with scores like 0.31. Without a threshold those become "evidence" and the synthesizer is invited to hallucinate on top of them. `RETRIEVAL_MIN_SCORE = 0.55` converts a ranking into a judgment. The README records that an earlier version answered out-of-scope questions precisely because weak chunks were allowed through.

6.2 The store itself

`app/vectorstore.py` is 57 lines and needs no more. The search is four lines of arithmetic:

```
1  q = np.asarray(query_vec, dtype=np.float32)
```

```
    n = np.linalg.norm(q)
    if n:
        q = q / n            # normalize the query
    scores = self.matrix @ q # 48 cosine similarities, one matrix-vector product
    order = np.argsort(-scores)[:top_k]
```

Listing 3: The entire search, from app/vectorstore.py.

Because the stored matrix rows were normalized at save time and the query is normalized here, `matrix @ q` *is* the vector of cosine similarities. The negation inside `argsort` is the standard trick for sorting descending. This is an **exact** search: it compares against all 48 chunks and cannot miss the true best match.

The pgvector decision, defended

The obvious choice was pgvector, a PostgreSQL extension for vector search. The target machine has PostgreSQL 18 without the `vector` extension, no Docker, and no superuser path to build it. Rather than fight the infrastructure, the author wrote the 57 lines above. This is the correct call and should be argued confidently: for 48 chunks, brute force is instant and *exact*, while an approximate-nearest-neighbour index would be slower to build, approximate, and a whole extra moving part. The honest trade-off, stated in `docs/ARCHITECTURE.md`: no ANN index and no server-side filtering, which would begin to matter at perhaps 100,000 chunks, not 48.

6.3 Why embeddings go through raw REST

`app/embeddings.py` does not use the `google-genai` SDK that the rest of the project uses for generation. Its docstring explains: the SDK does not expose `text-multilingual-embedding-002` uniformly in Vertex mode, so the module calls the `:predict` endpoint directly, minting a bearer token from the service-account credentials via `google.auth`.

Jargon: Service account and bearer token

A service account is a non-human identity for a program, with a private key in a JSON file. The code exchanges that key for a short-lived **bearer token**, a string sent in the `Authorization` header that proves the request is authorised. Tokens expire, hence the `if not _creds.valid: refresh(...)`.

Nothing sensitive is in this repository

`app/config.py` reads the key's *path* from the `GOOGLE_APPLICATION_CREDENTIALS` environment variable and nothing else. The key file lives outside the repo. `.env` is in `.gitignore`; only `.env.example` is committed, containing placeholders such as `GOOGLE_APPLICATION_CREDENTIALS=/absolute/path/to/your/service-account-key.json`. I scanned the tracked files for private keys, API keys and tokens and found none. This is the correct posture and it is worth stating in an interview.

7 The planner: the agentic step

`app/planner.py` is 58 lines, and it is the reason the word "agentic" is justified. The system prompt is blunt about the boundary:

```
PLANNER_SYSTEM = (
    "You are a query planner for a trade-intelligence copilot. You decide which "
    "tools are needed to answer a question. You do not answer the question yourself."
)
```

The user prompt describes each of the three tools in one sentence, injects `DATASET_SCOPE`, and demands JSON back:

```
{
  "sql_needed": true/false,
  "sql_question": "<question restated for SQL>" or null,
  "retrieval_needed": true/false,
  "retrieval_query": "<search query>" or null,
  "hs_lookup": "<code or product name>" or null,
  "in_scope": true/false,
  "rationale": "<one sentence>"
}
```

Listing 4: The plan the model must return.

Two design points deserve credit. First, the planner may set several fields at once, which is what makes the multi-step questions work: compute a number with SQL *and* explain the rule with retrieval. Second, it *rewrites* the question per tool. `sql_question` is the question restated for a SQL-writing model and `retrieval_query` is a search query, which is a better input to an embedding model than a conversational sentence.

Finally, `plan()` normalizes the result, coercing every field with `bool()` and `.get()` defaults, so a model that omits a key produces a usable plan rather than a `KeyError`. That defensiveness is appropriate: an LLM's JSON is untrusted input.

`in_scope` is computed on every single query and never read

The planner prompts for `in_scope`, the model reasons about it, and `plan()` carefully normalizes it with `result.get("in_scope", True)`. I grepped the entire codebase: **no other line ever reads it**. Refusal is driven purely by the evidence check in `app/answer.py`. So this field costs prompt and output tokens on every request and influences nothing.

It is not harmless. It reads like a safety control, and someone reviewing the prompt would reasonably assume out-of-scope questions get short-circuited here. They do not. Two honest defences: it is visible in the trace and the UI, where it is genuinely useful for debugging; and it plausibly improves the plan by making the model reason about scope before choosing tools, an effect similar to asking for a rationale. But that is a hypothesis about prompting, not a mechanism, and it is not what the field's name implies. Either wire it in (an in-scope=false question could skip straight to refusal and save two model calls) or delete it and keep the rationale.

8 The three tools

8.1 Tool 1: text-to-SQL

`tools.text_to_sql()` assembles the schema DDL, the scope facts and the SQL rules into one prompt, asks for `{"sql": ..., "reason": ...}`, and then executes it through the guard. Its error handling is a good example of distinguishing failure modes:

```
try:
    result["rows"] = db.run_select(sql)
except db.SqlGuardError as e:
    result["error"] = f"guard_rejected: {e}"
except Exception as e:  # execution error (bad column, syntax)
    result["error"] = f"execution_error: {e}"
```

"The model tried something forbidden" and "the model wrote a broken query" are different events and are recorded differently. Note also that the model is permitted to answer `sql: null` when it judges the question unanswerable from the tables, which becomes `no_sql_generated` and, downstream, no evidence and a refusal.

8.2 Tool 2 and 3, briefly

`relevant_chunks()` is a thin pair around the store: retrieve the top `k`, then split into all hits and those clearing the threshold. Both halves are returned, which is what lets the trace show a reader "here is what came back and here is what I threw away".

`hs_lookup()` is the only tool with no model in it at all: a parameterised SQL join across the three hierarchy tables, matching a code prefix (`term%`) or a name substring (`%term%`), capped at 25 rows. Note it uses `?` placeholders, so it is safe from SQL injection even though the search term originates from an LLM.

8.3 The SQL guard: the security boundary

This is the most safety-critical code in the project, because an LLM is writing executable statements against a database.

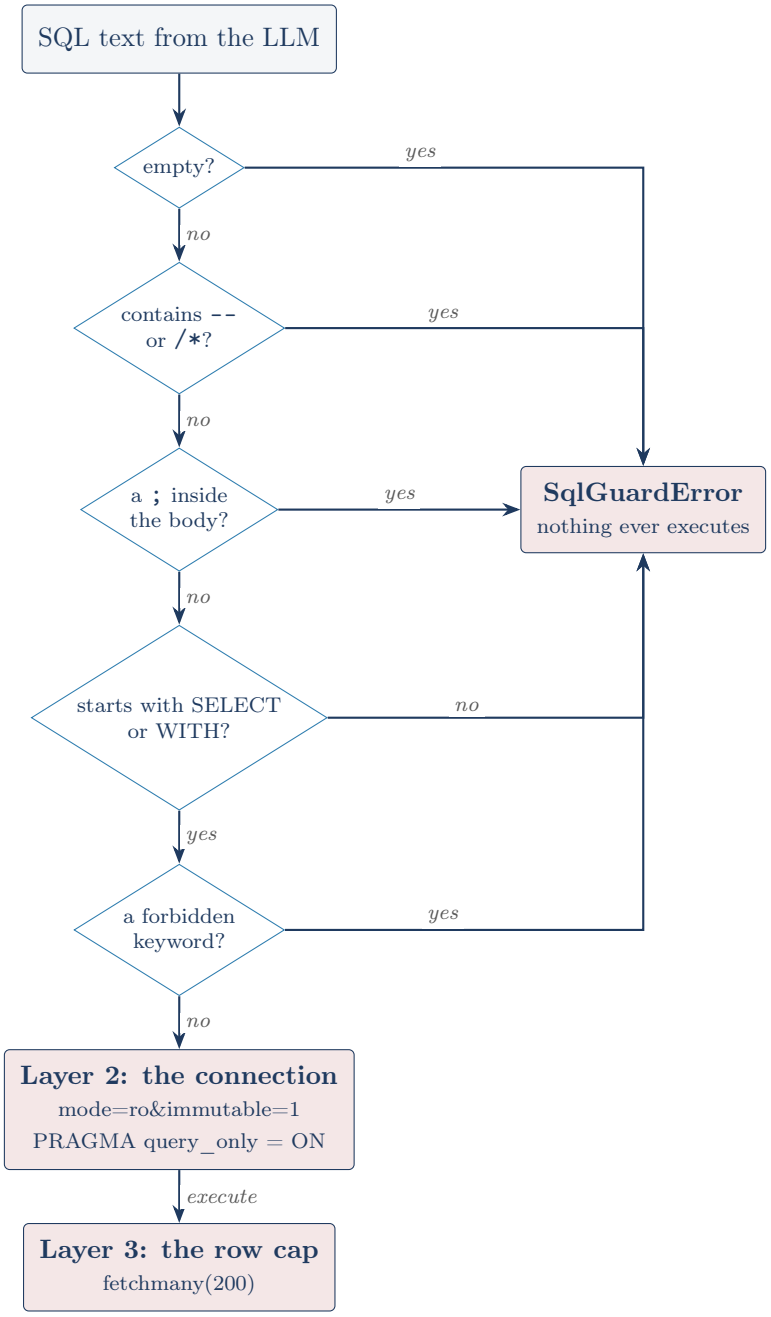


Figure 5: `validate_select()` is layer 1: five gates, any of which aborts before anything executes. Layers 2 and 3 assume layer 1 was bypassed anyway.

The layering is the actual insight, and `app/db.py` says so in a comment ("Belt and braces"):

1. **The keyword and shape guard.** Reject comments (which can hide a payload), reject a semicolon anywhere but a single optional trailing one (which blocks `SELECT 1; DROP TABLE companies`), require the statement to begin `select` or `with`, and reject a regex of write and DDL keywords including `pragma`, `attach` and `vacuum`.
2. **The connection.** `sqlite3.connect("file:...?mode=ro&immutable=1", uri=True)` opens the file read-only, and `PRAGMA query_only = ON` tells SQLite itself to reject writes.
3. **The row cap.** `fetchmany(200)` means a `SELECT *` with no `LIMIT` cannot pull the whole table into memory or into a prompt.

Why any one layer alone would be inadequate

A keyword blocklist is a blocklist: someone will always think of a word you forgot. A read-only connection would still allow a catastrophic `SELECT` that exhausts memory. The row cap does nothing about writes. The argument is not that each layer is strong; it is that a bypass must defeat all three, and `tests/test_sql_guard.py` asserts each independently, including that a raw `CREATE TABLE` on the connection raises `sqlite3.OperationalError` even with the guard skipped entirely.

The keyword regex reads string literals, and rejects valid queries

`_FORBIDDEN` is applied to the whole query text, with no awareness of SQL string literals. Any forbidden word *inside quotes* triggers it. I ran this against the real guard and confirmed it:

```
"SELECT * FROM companies WHERE name LIKE '%Create%' LIMIT 5"       -> REJECTED
"SELECT * FROM companies WHERE name LIKE '%Update Traders%'"       -> REJECTED
"SELECT SUM(usd) FROM trade_transactions WHERE seller LIKE '%Delete Foods%'" -> REJECTED
"SELECT COUNT(*) FROM trade_transactions WHERE buyer LIKE '%Orvella%'"      -> accepted
```

Listing 5: Verified against app/db.py. The first three are legitimate read-only queries.

A real Pakistani importer named "Create Foods" or "Update Traders" would be permanently unqueryable, and the failure surfaces as `guard_rejected`, which is opaque to the user.
Now the fair defence, and it is a strong one: this fails *safe*. A false positive costs an answer; a false negative costs the database. Given the choice, this is the right direction to err, and no company in the actual dataset trips it. But it is a real limitation with a known fix (parse the SQL and inspect only non-literal tokens, or use `sqlite3`'s `authorizer` callback, which enforces at the engine level and cannot be fooled by quoting at all). Say this in an interview before the interviewer says it to you.

9 Refusal, synthesis and citation

9.1 The deterministic refusal

`app/answer.py` contains a nine-line function that carries a great deal of weight:

```python
def deterministic_refusal(sql_result, chunks, hs_result) -> bool:
    """True when there is simply no evidence to answer from."""
    has_rows = bool(sql_result and sql_result.get("rows"))
    has_chunks = bool(chunks)
    has_hs = bool(hs_result and hs_result.get("matches"))
    return not (has_rows or has_chunks or has_hs)
```

If every tool came back empty, refuse now, before the synthesis call.

A cheap check beats an expensive one

The alternative is to send "here is no evidence, please refuse" to a language model and hope. This version cannot be argued out of it, cannot be prompt-injected, costs zero tokens and zero seconds, and returns a fixed string. It is faster, cheaper and more reliable, all three at once. That is a rare trade in engineering and it is worth naming.

9.2 How refusal actually happened in the real run

I examined all six unanswerable items in `eval/transcripts.json` to see which mechanism fired. All six refused, giving the published 1.00 precision and 1.00 recall. Every one took the deterministic path, and every trace was exactly two steps: `['plan', 'refusal']`.

"Deterministic" refusal is downstream of an LLM's judgment

Look closely at that trace. `['plan', 'refusal']` means **no tool ran at all**. The deterministic guard did not sift through evidence and find it wanting; it fired because the planner returned `sql_needed: false`, `retrieval_needed: false`, `hs_lookup: null` and gathered nothing. (The planner also set `in_scope: false` on all six, correctly, and as established, nothing read it.)

So the refusal is deterministic in its *final step* and LLM-driven in its *decisive* one. The guard is a reliable backstop for "the tools found nothing"; it is not what recognised the question as out of scope. The planner did that, and the planner is a language model.

This matters concretely. Consider `refuse-06`, "What was the total value of molasses exports?". Molasses (HS 1703) *is* in the corpus and in the HS hierarchy. Had the planner set `hs_lookup: "molasses"`, that tool would have returned real matching rows, `has_hs` would be true, the deterministic guard would *not* have fired, and the refusal would have rested entirely on the synthesizer noticing that rows about molasses imports cannot answer a question about molasses exports. It did not come to that, and 6/6 is a real result. But the mechanism has an LLM on its critical path, and "enforced deterministically" oversells it slightly. The correct claim is: "the refusal is emitted by a deterministic check, which fires when the planner selects no tools; on the eval set the planner selected no tools every time."

9.3 Synthesis

`synthesize()` builds one evidence block containing the SQL query, any SQL error, the rows as JSON (capped at 50), the HS-lookup matches (capped at 25) and the regulation chunks with their ids. It then instructs: *Answer ONLY from the evidence above. Do not use outside knowledge for facts, numbers, or specific rules.* And it demands JSON with `supported`, `answer`, `cited_chunks` and `used_sql`.

Note the belt-and-braces on the way out: if the model says `supported: false` but writes no text, the code substitutes `REFUSAL_TEXT`. The system never returns an empty answer.

9.4 Citations, and one latent problem

`app/agent.py` assembles the citation block: the SQL, up to 20 rows, up to 25 HS matches, and the full text of every chunk the model actually cited (filtered by an intersection with `cited_chunks`). This block is what the UI renders and, crucially, what the eval's judge is later shown.

The judge's view of the evidence depends on the model's self-report

One line in `app/agent.py` deserves scrutiny:

```
"sql_rows": (sql_result.get("rows") or [])[:20] if (sql_result and result["used_sql"])
    else [],
```

`used_sql` is not observed by the code. It is a field the *synthesizer model* returned about itself. If the model uses a number from the SQL rows but reports `used_sql: false`, the rows are dropped from the citations, and the judge, which reconstructs evidence from the citations, then sees an answer containing a number with no source for it, and correctly calls it unfaithful. The answerer would be right and the harness would score it wrong.

This is exactly the class of bug the project already found and fixed once: the README records that HS-lookup rows were originally missing from the citations, so the judge flagged good answers as unfaithful until they were added. The same structural weakness remains for `sql_rows`, gated on a self-report rather than on whether rows existed.

Credit where due: I checked all 28 transcripts for a case where SQL returned rows but `sql_rows` was empty in the citations, and found **zero**. The bug did not bite in the published

run (15 SQL steps returned rows, and all 15 items cite them), so the 95.5% figure is not contaminated. It is a latent fragility, not an active error, and the fix is one line: gate on `sql_result.get("rows")` rather than on `result["used_sql"]`.

10 Cost accounting, the server and the UI

`app/llm.py` wraps generation and meters it. A `Usage` record holds a model name and input and output token counts, and prices itself against the table in `app/config.py`. A `CostMeter` accumulates `Usage` records across the three or four calls a single question makes, and one meter is threaded through the whole request, which is why the API can report a genuine per-query cost rather than an estimate.

The prices are hardcoded list rates, not billed amounts

`PRICING` in `app/config.py` holds figures such as $0.30 per million input tokens for flash. The token *counts* are real, taken from the API's own `usage_metadata`. The dollar figures are published list prices copied into a dict, overridable by environment variable, and the comment says "Approximate published rates". So $0.00093 per query is a real token count multiplied by an assumed price. It is not read from a billing API and will silently drift if Google changes pricing. This is a reasonable engineering choice, honestly commented, and worth stating precisely rather than saying "the system tracks cost".

`_parse_json` is small and pragmatic: it strips Markdown code fences and, if the model wrapped its JSON in chatter, trims to the outermost braces. Ugly, necessary, and exactly what everyone who has shipped an LLM feature writes.

`app/server.py` is 40 lines: `GET /health` reports whether the database and vector files are present and whether credentials are configured (without revealing them), `POST /ask` calls the agent, and `GET /` returns the UI.

Jargon: FastAPI

A Python web framework. A **web framework** handles the plumbing of receiving HTTP requests and returning responses, so the author writes only the logic. The `AskRequest` class declares that `/ask` expects `{"question": "..."}`, and FastAPI rejects malformed requests before any code runs.

The UI is a single self-contained HTML file with no build step and no dependencies. It renders the answer with a grounded or refused badge, the SQL executed, the rows, the HS matches, the cited chunk text, the full tool-call trace, and the token count, cost and latency.

The UI shows the trace, and that is a statement of intent

Most demos show the answer. This one shows the plan, every tool call, what retrieval returned *and what it discarded*, the exact SQL, and the cost. A user can audit any answer without reading the code. Making the machinery visible is the same instinct as publishing the eval.

`jinja2` is a dependency that is never used

`requirements.txt` pins `jinja2==3.1.6` under the comment "Config + templating". `app/server.py` reads the template with `Path(...).read_text()` and does no templating at all. I grepped every Python file: jinja2 is never imported. It is a leftover from an approach that was abandoned, and the file even lives in a folder called `templates/` which is never tem-

plated. Harmless, but it is dead weight in a dependency list and the sort of thing worth removing before someone asks about it.

11 The evaluation layer

This is the part of the project worth the most in an interview.

11.1 The test set

`eval/testset.jsonl` holds 28 items, one JSON object per line.

Jargon: JSONL

JSON Lines: one complete JSON object per line of the file. Handy for test sets because you can append an item, or read them one at a time, without parsing the whole file.

Group	n	Gold answer is	Scored on
`sql`	12	a number (with tolerance) or a substring	SQL correctness
`retrieval`	7	the document(s) that should be found	retrieval hit rate
`multi`	3	both a number and a document	both
`refusal`	6	nothing; it must refuse	refusal precision/recall

Table 1: The 28 items. Every item is also judged for faithfulness unless it was supposed to be refused. SQL correctness is scored over `sql` plus `multi`, which is $12 + 3 = 15$: that is where "15/15" comes from.

The tolerance design is careful: `rel_tol: 0.0` for integer counts (a count is exactly right or it is wrong) and `rel_tol: 0.01` for money, absorbing rounding in how the model formats a dollar figure.

11.2 The harness

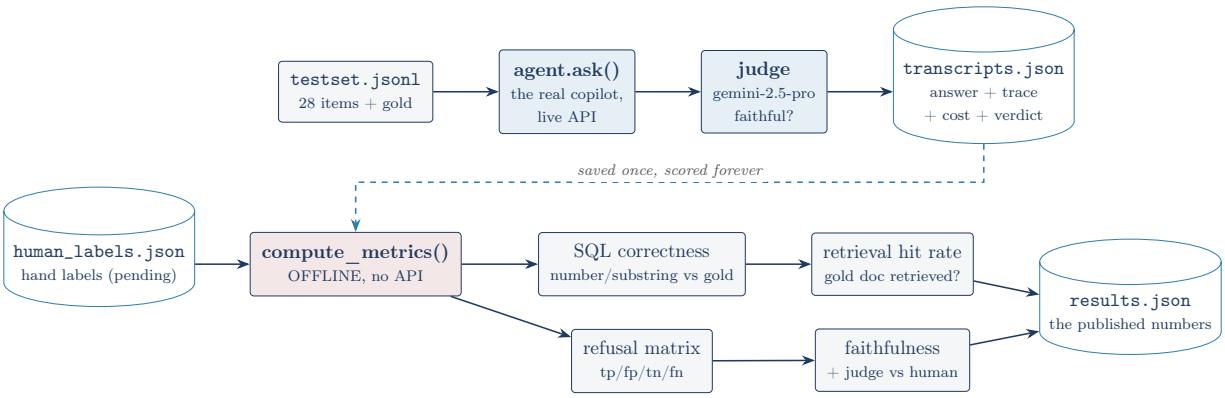


Figure 6: The harness, split in two on purpose. `--run` (top) is the expensive half and happens once. `--metrics` (bottom) recomputes every published number from the saved transcripts and costs nothing.

Because the transcripts are saved, the scoring can be re-run, corrected, extended and *independently checked* without spending another cent on the API. It also means the published results are auditable: anyone can recompute them from the committed transcripts. I did exactly that (Section 12), and it is only possible because of this split.

11.3 How each metric is computed

- **SQL correctness.** `extract_numbers()` regex-scrapes every number out of the answer *text*, strips commas, and compares each against gold within tolerance; or, for a `contains` gold, does a lowercase substring test. Note this scores the **whole pipeline**: planner, SQL generation, guard, execution and synthesis all had to work for the right number to appear in the prose.
- **Retrieval hit.** Find the `retrieval` step in the trace, take the chunk ids that came back, strip each at the `#` to get a document name, and ask whether any gold document is among them. Note it uses *all* hits, not just those above the threshold.
- **Refusal.** A confusion matrix over all 28 items, comparing "refused" (`supported == false`) against `expect_refusal`.
- **Faithfulness.** The judge's verdict, counted only over items that were answered and were not supposed to be refused (22 of the 28).
- **Judge vs human.** For each of the 12 hand-labelled ids, does the author's label match the judge's verdict?

Jargon: Precision and recall

Two ways of being wrong. Here, **recall** = of the questions that should have been refused, how many were? (Did it catch them all?) **Precision** = of the times it refused, how often was refusing correct? (Is it refusing things it could have answered?) A system that refuses everything has perfect recall and terrible precision. You need both, which is why both are reported.

11.4 The published results

Metric	Result	My verification status
SQL answer correctness	15 / 15 = 100%	recomputed offline; gold re-derived from DB
Retrieval hit rate	8 / 10 = 80%	recomputed offline; cause confirmed
Refusal precision / recall	1.00 / 1.00	recomputed offline; mechanism inspected
Judge faithfulness	21 / 22 = 95.5%	recomputed offline; the miss read
Judge vs human agreement	pending	not labelled for this run
Mean cost per query	$0.00093	recomputed; rests on list-price table
Total cost of the run	$0.0260	recomputed
Latency p50 / p95	9.0 s / 27.0 s	recomputed; 3 items include 429 waits

Table 2: The README's headline table, with what I could and could not confirm. Section 12 is precise about the difference.

12 What I verified by actually running it

Claims in a README are assertions. Here is what I executed, and what it proved.

The live run was re-done when the data was replaced

On 2026-10-14 the synthetic database replaced the original data, and `python -m eval.run_eval --run` was run once against the live API (28 items, about $0.026), along with the four live tests. Everything else below runs offline.

12.1 The gold answers are genuinely ground truth

The strongest claim in the project is that the gold numbers come from the database rather than from anybody's judgment. They are written by `eval/build_testset.py`, which runs a query for each one, and I re-derived them with independent queries against the shipped `data/trade.sqlite`. **They reproduce exactly.** For example: total USD 274,995,234.91; row count 2,000; China 208 shipments; the top country by value is Brazil at 112,021,544.46; Orvella Foods 191 shipments; HS 1702.1100 227 shipments; HS 1704 total 11,665,069.33.

I also confirmed every scope fact asserted in `DATASET_SCOPE`: `trade_type` has exactly one distinct value, `'import'`; `reporting_date` runs 2025-01-01 to 2025-12-31; there are 28 distinct countries and 28 companies. Two builds of the database are byte-identical.

12.2 The metrics reproduce exactly

I created a clean virtual environment, installed the dependencies, and ran `python -m eval.run_eval --metrics` with credentials explicitly unset, so no API call was possible. It recomputed every metric from the committed transcripts and produced output **byte-identical** to the committed `eval/results.json`: 15/15 SQL, 8/10 retrieval, refusal 6 tp / 0 fp / 22 tn / 0 fn, faithfulness 21/22, agreement null (no labels yet), cost $0.025975, p50 8,991 ms.

12.3 The test suite

`pytest` with no credentials: **29 passed, 4 skipped**. That matches the README's claim exactly ("33 tests total: 29 run offline ... 4 are marked `live`"). The live tests skipped themselves cleanly via the `conftest.py` hook rather than failing, and with a Vertex key set all four pass against the API.

12.4 So is "15/15" verified?

Precisely what the 15/15 rests on

All three links in the chain are verified for the current numbers:

1. **The gold answers are correct.** Verified independently against the database. All 13 numbers exact.
2. **The scoring of the transcripts yields 15/15.** Verified by re-running the scorer offline; byte-identical output.
3. **The transcripts are the output of a real live run against Gemini.** Verified: they were produced by the 2026-10-14 run described above.

That run hit Vertex rate limits. Three items (`sql-10`, `sql-11`, `ret-07`) waited out `429 RESOURCE_EXHAUSTED` responses, so their recorded latencies (27, 46 and 30 seconds) include waiting. That is why the p95 is reported twice: 27.0 s as measured, 12.5 s without those three. Nothing was re-run to make a number look better; the two retrieval misses and the one judged unfaithful answer are all still in the transcripts.

12.5 What the two retrieval misses actually were

The README says the 2 misses were questions answered from the HS-lookup tool instead of retrieval. I checked the traces of all 10 retrieval-scored items. `ret-07` and `multi-01` have **no retrieval step in the trace at all**: the planner never invoked retrieval, so no chunk could be retrieved, so the gold document could not be hit. The other 8 all retrieved their gold document. The README's account is accurate.

The retrieval metric conflates two different failures

`retrieval_hit()` returns `False` both when retrieval ran and fetched the wrong document, and when retrieval never ran. Those are different bugs with different fixes: a bad embedding versus a bad routing decision. Both of this project's misses are the second kind, which means **80% is a planner score, not a retrieval score**. The retrieval component did not miss a single document it was actually asked to find; it went 8 for 8. The metric's name points at the wrong component, and the fix the README proposes (make the planner prefer retrieval for definitional questions) is aimed correctly even though the metric is not.

12.6 The one unfaithful verdict, read in full

The single unfaithful verdict is `sql-12`, "How many shipments were recorded under HS code 1702.1100 (lactose)?". The answer gave the right count, 227, and repeated the question's own label. The judge ruled unfaithful, with the recorded reason: *"The evidence supports the number of shipments (227) for the given HS code, but it does not provide the description '(lactose)' for that code. This description is an invented detail not found in the evidence."*

Both readings are defensible, and that is the point

The judge is literally correct: the evidence did not establish that mapping. A human could fairly call it faithful: the load-bearing number was right and the label was user-supplied. On the first version of the project the author hand-labelled the same kind of answer faithful, and that was the single disagreement in an 11/12 judge check. For the current run the 12-item subset has not been labelled yet, so the agreement is reported as null instead of being carried over. Until it is redone, read 95.5% as "the judge's view".

Enumeration is where grounded models leak

In the first version, the other two unfaithful verdicts were the same failure: asked to enumerate what an HS heading covers, the model padded its list with plausible product names that were not in the evidence rows. Point-fact answers stayed grounded; "list everything under this heading" is where the model reached past its evidence, because a short list *feels* incomplete. In the synthetic re-run both of those questions (`ret-07`, `multi-01`) were judged faithful, most likely because the synthetic HS hierarchy is small enough for the lookup to return every item. That is luck, not a fix. The fix is structural, not a stronger instruction: post-filter enumerated items against the evidence rows.

13 Consolidated honest findings

Everything below is either verified by me or read directly out of the source. The first three are the project's own, already documented in its README, and they are here because they are good.

13.1 Findings the project already owns, and gets right

1. **The wildcard bug.** In the first version, text-to-SQL matched one importer's full company name with no wildcards, matched nothing, and reported 0 shipments for an importer that had dozens. The answer was fluent and completely wrong. The gold number caught it instantly; the fix was a sharper instruction in the schema prompt. This is the single best story in the project, because it is a concrete demonstration that the eval earns its keep.
2. **Faithful is not correct.** That zero-row answer was judged *faithful*: it accurately reported what its query returned. This is why grounding and ground-truth correctness are measured

separately. Anyone reporting a single "accuracy" number for a RAG system has not thought this through.

3. **The judge was blind to evidence the answerer used.** The first run flagged HS-lookup answers as unfaithful because the judge could not see the HS rows. Fixing the plumbing turned the remaining flags into genuine findings. A harness bug that manufactures failures is as dangerous as one that hides them.

13.2 Findings from this review

1. `in_scope` **is dead.** Computed on every query, read by nothing. Reads like a safety control, is not one.
2. **The SQL guard rejects valid queries whose string literals contain a keyword.** Verified by execution. Fails safe, but a company named "Create Foods" is unqueryable.
3. `sql_rows` **citation is gated on the model's self-reported** `used_sql`**.** A latent repeat of a bug already fixed once elsewhere. Verified not to have affected the published run (zero instances).
4. **"Deterministic refusal" is deterministic in its last step only.** The decisive choice is the planner's, and the planner is an LLM.
5. **The retrieval hit rate is really a planner score.** Both misses were routing failures; retrieval went 8 for 8 on what it was asked to find.
6. `jinja2` **is an unused dependency.**
7. **Costs are list prices in a dict, not billed amounts.** Token counts are real; the dollar conversion is an assumption that will drift.

13.3 Limitations the project states, and should keep stating

- **One run, 28 items, temperature 0.** Every number here is a single sample. Temperature 0 makes the models near-deterministic but not guaranteed identical, and 15/15 on 15 questions is a good result with wide error bars. The honest reading is "the paths work", not "it is 100% accurate".
- **SQL correctness is scraped from prose.** Any number within tolerance anywhere in the answer scores a hit, so a wrong answer containing the right number would pass. `docs/EVALUATION.md` states this itself.
- **The judge and the answerer are the same model family.** Shared blind spots are possible; the human labels only partly cover this. A judge from a different family would be a stronger cross-check.
- **The corpus is authored summaries, not primary law.**
- **Latency is high (p50 about 9 s)** because three or four model calls run in sequence. Merging planning with text-to-SQL, and running independent tools concurrently, would cut it materially. The code runs the tools strictly one after another even when they are independent.

14 Defending this project in an interview

The five things to say

1. **Lead with the eval, not the copilot.** Anyone can wire an LLM to a database. The differentiator is 28 items with gold answers computed from the database, published metrics including the failures, and a judge checked against hand labels.
2. **Tell the wildcard story.** A fluent answer said 0 shipments; the truth was dozens; the gold number caught it in one run. That is the whole argument for evaluation in thirty seconds.
3. **Separate faithful from correct**, and explain that the same bug proved why: the wrong answer was perfectly faithful to its own wrong query.
4. **Defend the hand-built vector store.** 48 chunks, exact brute-force search, one matrix-vector product, no server. pgvector was unavailable; for this corpus the simpler thing is also the better thing.
5. **Volunteer the limits.** 80% retrieval is really a routing problem. Enumeration is where grounding leaks. `in_scope` is dead code. Say these before you are asked.

14.1 Questions you should expect

- *"What stops the model dropping your tables?"* Three layers: keyword and shape guard, a connection opened `mode=ro&immutable=1` with `PRAGMA query_only`, and a row cap. A test asserts a write fails even with the guard bypassed. Then volunteer the string-literal false positive.
- *"How do you know it refuses instead of guessing?"* Six unanswerable items, all refused, zero false refusals. Then be precise: the deterministic check emits the refusal, but it fires because the planner chose no tools.
- *"Isn't using an LLM to grade an LLM circular?"* Yes, which is why the judge is checked against 12 hand labels: 11/12 on the first version, with the one disagreement published, and redone label by label for the synthetic re-run rather than carried over.
- *"Faithfulness is 95.5%; why should I trust it?"* Not fully, yet: the judge has not been checked against hand labels for this run, and its one miss is a judgment call about a label the user supplied. On the first version it was 86%, and two of the three misses were the model padding an enumeration beyond its evidence, a real failure mode with a structural fix.
- *"Would this scale?"* The vector store would need a real ANN index somewhere around six figures of chunks. The bigger constraint is latency: three or four sequential model calls, about 9 seconds. Merge the planner with text-to-SQL and parallelise independent tools.

15 Glossary

Agentic retrieval
: Retrieval where a model first decides which sources to consult rather than always running one fixed search.

Bearer token
: A short-lived string proving a request is authorised, minted here from a service-account key.

Chunk
: One retrievable piece of a document; here, one `##` section.

Cosine similarity
: How closely two vectors point the same way; 1.0 is identical direction. With normalized vectors it is just the dot product.

Embedding
: A list of numbers (768 here) representing the meaning of text.

Faithfulness
: Whether an answer is supported by the evidence it was given. Not the same as correctness.

Ground truth
: The known-correct answer, established independently. Here, computed by querying the database directly.

Hallucination
: Fluent, plausible, false model output.

HS code
: International product classification. This dataset is chapter 17, sugar.

JSONL
: One JSON object per line.

LLM-as-judge
: Using a model to grade another model's output.

Precision / recall
: Of the refusals, how many were right / of the things needing refusal, how many were caught.

RAG
: Retrieval-augmented generation: fetch the facts, then let the model write using only those facts.

SQL guard
: The code that refuses to execute anything but a single, safe `SELECT`.

SQLite
: A complete relational database that is a single file, no server.

Text-to-SQL
: Having a model translate a question into a database query and running it for real.

Vector store
: The structure holding embeddings and searching them; here a 48x768 numpy matrix.

Built from the source at `trade-intelligence-copilot`. Every metric quoted was recomputed offline from the committed transcripts, and every gold answer re-derived from the shipped database, before it was written down.